C112156233 蘇泓叡 LLMs

Week1

This document records the questions I asked AI and the assistance I received while completing this project.

Question List

During the development of this project, I asked AI the following four core questions:

1. How to extract RGB information from an image
2. How to iterate through that information
3. How to use the K-means algorithm in scikit-learn
4. Help me write a function to compare different K values and display the results

Questions 1 and 2: Image Loading and Data Processing

Through the discussion of the first two questions, I obtained the following code:

```
def load_image(image_path):
      """載入圖片並轉換為 numpy 陣列"""
      img = Image.open(image_path)
      img_array = np.array(img)
      return img_array
```

This function is responsible for reading image files and converting them into numpy array format for subsequent processing.

Question 3: K-means Image Compression Implementation

Based on the answer to the third question, I obtained the core image compression function:

```
def compress_image(img_array, k):
      """使用 K-means 壓縮圖片"""
      # 獲取圖片的形狀
      h, w, c = img_array.shape
      # 將圖片重塑為 (pixels, channels)
      pixels = img_array.reshape(-1, c)
      # 進行 K-means 聚類
      kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
      kmeans.fit(pixels)
      # 用聚類中心替換原始像素值
      compressed_pixels = kmeans.cluster_centers_[kmeans.labels_]
      # 重塑回原始圖片形狀
      compressed_img = compressed_pixels.reshape(h, w, c)
      # 確保像素值在 0-255 範圍內
```

```
    compressed_img = np.clip(compressed_img, 0, 255).astype(np.uint8)
    return compressed_img
```

Core Concept Explanation:

The two functions above are the core components of this program:

- `load_image()` handles data input
- `compress_image()` implements the K-means clustering algorithm to compress image colors

Question 4: K-Value Comparison and Visualization

Finally, I asked AI to help me write a function to compare the effects of different K values and display the results:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from PIL import Image
import os
def load_image(image_path):
    """載入圖片並轉換為 numpy 陣列"""
    img = Image.open(image_path)
    img_array = np.array(img)
    return img_array
def compress_image(img_array, k):
    """使用 K-means 壓縮圖片"""
    # 獲取圖片的形狀
    h, w, c = img_array.shape
    # 將圖片重塑為 (pixels, channels)
    pixels = img_array.reshape(-1, c)
    # 進行 K-means 聚類
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(pixels)
    # 用聚類中心替換原始像素值
    compressed_pixels = kmeans.cluster_centers_[kmeans.labels_]
    # 重塑回原始圖片形狀
    compressed_img = compressed_pixels.reshape(h, w, c)
    # 確保像素值在 0-255 範圍內
    compressed_img = np.clip(compressed_img, 0, 255).astype(np.uint8)
    return compressed_iProcess video to extract feature
I got this code by coping and modifying Medaipipe official guide:

def extract_features(video_path, label, frame_interval):
•
    # Eye gaze features (8 features)
    eye_gaze_features = [
        'eyeLookUpLeft', 'eyeLookUpRight',
        'eyeLookDownLeft', 'eyeLookDownRight',
        'eyeLookInLeft', 'eyeLookInRight',
```

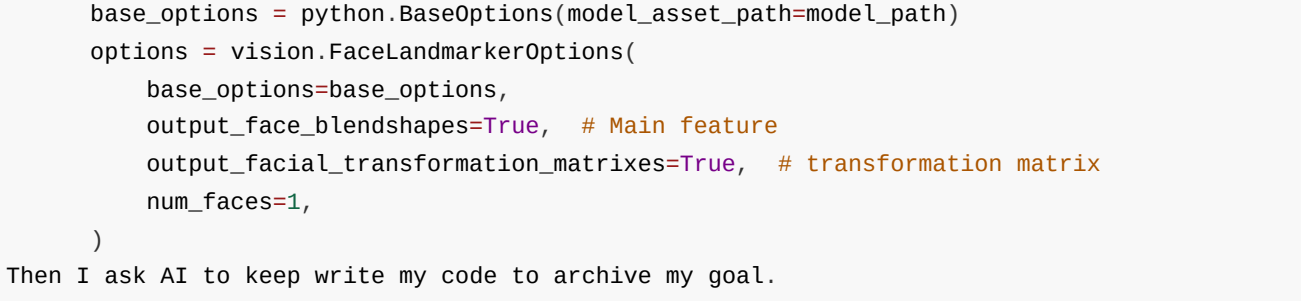
```
            'eyeLookOutLeft', 'eyeLookOutRight'
        ]

        # Eye blink features (2 features)
        eye_blink_features = ['eyeBlinkLeft', 'eyeBlinkRight']


        logging.info(f"Start extract feture from video {video_path}")
        # Create FaceLandmarker

        base_options = python.BaseOptions(model_asset_path=model_path)
        options = vision.FaceLandmarkerOptions(
            base_options=base_options,
            output_face_blendshapes=True,  # Main feature
            output_facial_transformation_matrixes=True,  # transformation matrix
            num_faces=1,
        )
```

Then I ask AI to keep write my code to archive my goal.



Then I got the remain code:

```
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    logging.info(f"Video FPS: {fps}, Total frames: {total_frames}")

    # Storage for extracted data
    data_rows = []

    frame_count = 0
    processed_count = 0

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        # Process every nth frame
        if frame_count % frame_interval != 0:
            frame_count += 1
            continue

        # Convert BGR to RGB
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)

        # Detect face landmarks
        detection_result = detector.detect(mp_image)

        # Initialize row data
        row_data = {
            'video_path': video_path,
```

```
            'label': label,
            'frame_number': frame_count,
            'timestamp': frame_count / fps
        }

        # Extract blendshapes
        if detection_result.face_blendshapes:
            blendshapes = detection_result.face_blendshapes[0]

            # Extract all blendshapes
            for blendshape in blendshapes:
                row_data[blendshape.category_name] = blendshape.score

        # Extract transformation matrix
        if detection_result.facial_transformation_matrixes:
            matrix = detection_result.facial_transformation_matrixes[0]

            # Flatten 4x4 transformation matrix
            # Matrix is typically 4x4 for homogeneous coordinates
            for i in range(len(matrix)):
                for j in range(len(matrix[i])):
                    row_data[f'transform_matrix_{i}_{j}'] = matrix[i][j]

        data_rows.append(row_data)
        processed_count += 1

        if processed_count % 100 == 0:
            logging.info(f"Processed {processed_count} frames")

        frame_count += 1

    cap.release()
    logging.info(f"Total frames processed: {processed_count}")

    # Convert to DataFrame and save
    if data_rows:
        df = pd.DataFrame(data_rows)
        df.to_csv(output_csv, index=False)
        logging.info(f"Saved features to {output_csv}")
        logging.info(f"Shape: {df.shape}")
        logging.info(f"Columns: {list(df.columns)}")
        return df
    else:
        logging.warning("No data extracted")
        return None
```

I note the matrix formate might not good to train model, so I ask AI modify them, to calculate the matrix to three value:


```
# Extract transformation matrix - convert to 3 features
if detection_result.facial_transformation_matrixes:
    matrix = detection_result.facial_transformation_matrixes[0]
```

```
    # Extract 3 values from transformation matrix
    # Using translation components (last column, first 3 rows)
    # These represent the 3D position/translation of the face
    row_data['transform_x'] = matrix[0][3] if len(matrix) > 0 and len(matrix[0]) > 3 else 0
    row_data['transform_y'] = matrix[1][3] if len(matrix) > 1 and len(matrix[1]) > 3 else 0
    row_data['transform_z'] = matrix[2][3] if len(matrix) > 2 and len(matrix[2]) > 3 else 0
```

I studied the snippet code and realized something wrong, the transformation matrix should be calculated to the value represent the rotation. So I ask AI to modify again.



```
# Extract head rotation from transformation matrix (pitch, yaw, roll)
if detection_result.facial_transformation_matrixes:
    # Get the 4x4 transformation matrix
    matrix = np.array(detection_result.facial_transformation_matrixes[0]).reshape(4, 4)

    # Extract rotation angles from rotation matrix (top-left 3x3)
    R = matrix[:3, :3]

    # Calculate Euler angles (pitch, yaw, roll) in radians
    pitch = np.arctan2(-R[2, 0], np.sqrt(R[2, 1]**2 + R[2, 2]**2))
    yaw = np.arctan2(R[1, 0], R[0, 0])
    roll = np.arctan2(R[2, 1], R[2, 2])

    row_data['head_pitch'] = pitch
    row_data['head_yaw'] = yaw
    row_data['head_roll'] = roll
```

I didn't know does AI work right because of my bad math, but let it go for now.

Then I paste the structure of dataset, ask AI to handle them:



Training SVM Classifier

After extracting all features from the videos, I need to train a classifier. I asked AI to write a training script for me.

I told AI that I want to use SVM with RBF kernel, and I need to:

Filter out frames where eyes are closed

Split data into training and testing sets

Perform hyperparameter tuning with GridSearchCV

Evaluate the model with 7 metrics (Confusion Matrix, Accuracy, Precision, Recall, F1 Score, ROC Curve, AUC)

AI wrote the complete train_svm.py script for me, which includes:

Data loading and preprocessing

```
Feature scaling with StandardScaler

GridSearchCV for hyperparameter tuning (C and gamma)

Model evaluation with all 7 metrics

Visualization generation (confusion matrix and ROC curve)

Model saving

The script worked very well and achieved 97.7% accuracy on the test set.

Real-time Testing Program
I was curious whether the model was overfitting, so I asked AI to write a real-time webcam
prediction program.

I asked AI to create a program that:

Loads the trained model and scaler

Captures frames from webcam

Extracts features in real-time using MediaPipe

Predicts whether I'm watching the screen

Displays the result with a nice UI

AI wrote the complete realtime_prediction.py script for me, which includes:

Real-time face detection and feature extraction

Prediction with smoothing (moving average)

Visual feedback with color-coded status

Probability display

Keyboard controls (quit, screenshot, reset)mg
def compare_k_values(image_path, k_values):
      """比較不同 k 值的壓縮結果"""
      # 載入原始圖片
      original_img = load_image(image_path)
      h, w, c = original_img.shape
      # 計算原始圖片的顏色數量
      pixels = original_img.reshape(-1, c)
      original_colors = lProcess video to extract feature
I got this code by coping and modifying Medaipipe official guide:

def extract_features(video_path, label, frame_interval):
•
      # Eye gaze features (8 features)
```

```
    eye_gaze_features = [
          'eyeLookUpLeft', 'eyeLookUpRight',
          'eyeLookDownLeft', 'eyeLookDownRight',
          'eyeLookInLeft', 'eyeLookInRight',
          'eyeLookOutLeft', 'eyeLookOutRight'
      ]
•
      # Eye blink features (2 features)
      eye_blink_features = ['eyeBlinkLeft', 'eyeBlinkRight']
•
•
      logging.info(f"Start extract feture from video {video_path}")
      # Create FaceLandmarker
•
      base_options = python.BaseOptions(model_asset_path=model_path)
      options = vision.FaceLandmarkerOptions(
          base_options=base_options,
          output_face_blendshapes=True,  # Main feature
          output_facial_transformation_matrixes=True,  # transformation matrix
          num_faces=1,
      )
```

Then I ask AI to keep write my code to archive my goal.



Then I got the remain code:

```
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    logging.info(f"Video FPS: {fps}, Total frames: {total_frames}")

    # Storage for extracted data
    data_rows = []

    frame_count = 0
    processed_count = 0

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        # Process every nth frame
        if frame_count % frame_interval != 0:
            frame_count += 1
            continue

        # Convert BGR to RGB
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)

        # Detect face landmarks
        detection_result = detector.detect(mp_image)
```

```
        # Initialize row data
        row_data = {
            'video_path': video_path,
            'label': label,
            'frame_number': frame_count,
            'timestamp': frame_count / fps
        }

        # Extract blendshapes
        if detection_result.face_blendshapes:
            blendshapes = detection_result.face_blendshapes[0]

            # Extract all blendshapes
            for blendshape in blendshapes:
                row_data[blendshape.category_name] = blendshape.score

        # Extract transformation matrix
        if detection_result.facial_transformation_matrixes:
            matrix = detection_result.facial_transformation_matrixes[0]

            # Flatten 4x4 transformation matrix
            # Matrix is typically 4x4 for homogeneous coordinates
            for i in range(len(matrix)):
                for j in range(len(matrix[i])):
                    row_data[f'transform_matrix_{i}_{j}'] = matrix[i][j]

        data_rows.append(row_data)
        processed_count += 1

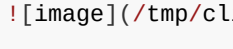
        if processed_count % 100 == 0:
            logging.info(f"Processed {processed_count} frames")

        frame_count += 1

    cap.release()
    logging.info(f"Total frames processed: {processed_count}")

    # Convert to DataFrame and save
    if data_rows:
        df = pd.DataFrame(data_rows)
        df.to_csv(output_csv, index=False)
        logging.info(f"Saved features to {output_csv}")
        logging.info(f"Shape: {df.shape}")
        logging.info(f"Columns: {list(df.columns)}")
        return df
    else:
        logging.warning("No data extracted")
        return None
```

I note the matrix formate might not good to train model, so I ask AI modify them, to calculate the matrix to three value:



```
# Extract transformation matrix - convert to 3 features
if detection_result.facial_transformation_matrixes:
    matrix = detection_result.facial_transformation_matrixes[0]

    # Extract 3 values from transformation matrix
    # Using translation components (last column, first 3 rows)
    # These represent the 3D position/translation of the face
    row_data['transform_x'] = matrix[0][3] if len(matrix) > 0 and len(matrix[0]) > 3 else 0
    row_data['transform_y'] = matrix[1][3] if len(matrix) > 1 and len(matrix[1]) > 3 else 0
    row_data['transform_z'] = matrix[2][3] if len(matrix) > 2 and len(matrix[2]) > 3 else 0
```

I studied the snippet code and realized something wrong, the transformation matrix should be calculated to the value represent the rotation. So I ask AI to modify again.



```
# Extract head rotation from transformation matrix (pitch, yaw, roll)
if detection_result.facial_transformation_matrixes:
    # Get the 4x4 transformation matrix
    matrix = np.array(detection_result.facial_transformation_matrixes[0]).reshape(4, 4)

    # Extract rotation angles from rotation matrix (top-left 3x3)
    R = matrix[:3, :3]

    # Calculate Euler angles (pitch, yaw, roll) in radians
    pitch = np.arctan2(-R[2, 0], np.sqrt(R[2, 1]**2 + R[2, 2]**2))
    yaw = np.arctan2(R[1, 0], R[0, 0])
    roll = np.arctan2(R[2, 1], R[2, 2])

    row_data['head_pitch'] = pitch
    row_data['head_yaw'] = yaw
    row_data['head_roll'] = roll
```

I didn't know does AI work right because of my bad math, but let it go for now.

Then I paste the structure of dataset, ask AI to handle them:

Training SVM Classifier

After extracting all features from the videos, I need to train a classifier. I asked AI to write a training script for me.

I told AI that I want to use SVM with RBF kernel, and I need to:

Filter out frames where eyes are closed

Split data into training and testing sets

Perform hyperparameter tuning with GridSearchCV

Evaluate the model with 7 metrics (Confusion Matrix, Accuracy, Precision, Recall, F1 Score, ROC Curve, AUC)

```
AI wrote the complete train_svm.py script for me, which includes:

Data loading and preprocessing

Feature scaling with StandardScaler

GridSearchCV for hyperparameter tuning (C and gamma)

Model evaluation with all 7 metrics

Visualization generation (confusion matrix and ROC curve)

Model saving

The script worked very well and achieved 97.7% accuracy on the test set.

Real-time Testing Program
I was curious whether the model was overfitting, so I asked AI to write a real-time webcam
prediction program.

I asked AI to create a program that:

Loads the trained model and scaler

Captures frames from webcam

Extracts features in real-time using MediaPipe

Predicts whether I'm watching the screen

Displays the result with a nice UI

AI wrote the complete realtime_prediction.py script for me, which includes:

Real-time face detection and feature extraction

Prediction with smoothing (moving average)

Visual feedback with color-coded status

Probability display

Keyboard controls (quit, screenshot, reset)en(np.unique(pixels, axis=0))
      # 設置子圖
      n_images = len(k_values) + 1
      cols = 3
      rows = (n_images + cols - 1) // cols
      fig, axes = plt.subplots(rows, cols, figsize=(15, 5 * rows))
      axes = axes.flatten() if n_images > 1 else [axes]
      # 顯示原始圖片
      axes[0].imshow(original_img)
      axes[0].set_title(
          f"original image\ncount of colors: {original_colors}\nsize: {h}x{w}",
```

````
            fontsize=10,
        )
        axes[0].axis("off")
        # 對每個 k 值進行壓縮並顯示
        for idx, k in enumerate(k_values, 1):
            print(f"正在處理 k={k}...")
            compressed_img = compress_image(original_img, k)
            axes[idx].imshow(compressed_img)
            axes[idx].set_title(f"K = {k}\ncount of colors: {k}", fontsize=10)
            axes[idx].axis("off")
        # 隱藏多餘的子圖
        for idx in range(n_images, len(axes)):
            axes[idx].axis("off")
        plt.tight_layout()
        plt.savefig("kmeans_compression_comparison.png", dpi=150, bbox_inches="tight")
        print("\n比較圖已保存為 'kmeans_compression_comparison.png'")
        plt.show()
    def main():
        # 使用 obama.webp 圖片
        image_path = "obama.webp"
        if not os.path.exists(image_path):
            print(f"錯誤: 未找到圖片 '{image_path}'")
            return
        # 要比較的 k 值列表
        k_values = [2, 4, 8, 16, 32, 64]
        print(f"\n開始 K-means 圖片壓縮比較")
        print(f"圖片路徑: {image_path}")
        print(f"K 值: {k_values}\n")
        # 執行比較
        compare_k_values(image_path, k_values)
    if __name__ == "__main__":
        main()
    `

## Summary

Through interactions with AI, I successfully built a complete K-means image compression
program. The program is capable of:
- Loading and processing image data
- Using the K-means algorithm for color clustering
  - Comparing the compression effects of different K values
  - Visualizing the compression results
  ``
````

Week 2

In this week's homework, I ask AI explain me `Agrawal Sigmoid`, it gave me a comprehensive answer that helped me understand the concept clearly.

Then I used it to fix the grammar mistakes I made in the homework, including this article, I think it's beneficial for me to have this kind of assistant to help various tasks.

Week 4

Homework 3 ChatGPT

Process video to extract feature

I got this code by coping and modifying Medaipipe official guide:

```
def extract_features(video_path, label, frame_interval):

      # Eye gaze features (8 features)
    eye_gaze_features = [
          'eyeLookUpLeft', 'eyeLookUpRight',
          'eyeLookDownLeft', 'eyeLookDownRight',
          'eyeLookInLeft', 'eyeLookInRight',
          'eyeLookOutLeft', 'eyeLookOutRight'
      ]

      # Eye blink features (2 features)
      eye_blink_features = ['eyeBlinkLeft', 'eyeBlinkRight']


      logging.info(f"Start extract feture from video {video_path}")
      # Create FaceLandmarker

      base_options = python.BaseOptions(model_asset_path=model_path)
      options = vision.FaceLandmarkerOptions(
          base_options=base_options,
          output_face_blendshapes=True,  # Main feature
          output_facial_transformation_matrixes=True,  # transformation matrix
          num_faces=1,
      )
```

Then I ask AI to keep write my code to archive my goal.

Then I got the remain code:

```
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    logging.info(f"Video FPS: {fps}, Total frames: {total_frames}")

    # Storage for extracted data
    data_rows = []

    frame_count = 0
    processed_count = 0

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break
```

```
        # Process every nth frame
        if frame_count % frame_interval != 0:
            frame_count += 1
            continue

        # Convert BGR to RGB
        rgb_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb_frame)

        # Detect face landmarks
        detection_result = detector.detect(mp_image)

        # Initialize row data
        row_data = {
            'video_path': video_path,
            'label': label,
            'frame_number': frame_count,
            'timestamp': frame_count / fps
        }

        # Extract blendshapes
        if detection_result.face_blendshapes:
            blendshapes = detection_result.face_blendshapes[0]

            # Extract all blendshapes
            for blendshape in blendshapes:
                row_data[blendshape.category_name] = blendshape.score

        # Extract transformation matrix
        if detection_result.facial_transformation_matrixes:
            matrix = detection_result.facial_transformation_matrixes[0]

            # Flatten 4x4 transformation matrix
            # Matrix is typically 4x4 for homogeneous coordinates
            for i in range(len(matrix)):
                for j in range(len(matrix[i])):
                    row_data[f'transform_matrix_{i}_{j}'] = matrix[i][j]

        data_rows.append(row_data)
        processed_count += 1

        if processed_count % 100 == 0:
            logging.info(f"Processed {processed_count} frames")

        frame_count += 1

    cap.release()
    logging.info(f"Total frames processed: {processed_count}")

    # Convert to DataFrame and save
    if data_rows:
        df = pd.DataFrame(data_rows)
```

```
        df.to_csv(output_csv, index=False)
        logging.info(f"Saved features to {output_csv}")
        logging.info(f"Shape: {df.shape}")
        logging.info(f"Columns: {list(df.columns)}")
        return df
    else:
        logging.warning("No data extracted")
        return None
```

I note the matrix formate might not good to train model, so I ask AI modify them, to calculate the matrix to three value:

```
# Extract transformation matrix - convert to 3 features
if detection_result.facial_transformation_matrixes:
    matrix = detection_result.facial_transformation_matrixes[0]

    # Extract 3 values from transformation matrix
    # Using translation components (last column, first 3 rows)
    # These represent the 3D position/translation of the face
    row_data['transform_x'] = matrix[0][3] if len(matrix) > 0 and len(matrix[0]) > 3 else 0
    row_data['transform_y'] = matrix[1][3] if len(matrix) > 1 and len(matrix[1]) > 3 else 0
    row_data['transform_z'] = matrix[2][3] if len(matrix) > 2 and len(matrix[2]) > 3 else 0
```

I studied the snippet code and realized something wrong, the transformation matrix should be calculated to the value represent the rotation. So I ask AI to modify again.

```
# Extract head rotation from transformation matrix (pitch, yaw, roll)
if detection_result.facial_transformation_matrixes:
    # Get the 4x4 transformation matrix
    matrix = np.array(detection_result.facial_transformation_matrixes[0]).reshape(4, 4)

    # Extract rotation angles from rotation matrix (top-left 3x3)
    R = matrix[:3, :3]

    # Calculate Euler angles (pitch, yaw, roll) in radians
    pitch = np.arctan2(-R[2, 0], np.sqrt(R[2, 1]**2 + R[2, 2]**2))
    yaw = np.arctan2(R[1, 0], R[0, 0])
    roll = np.arctan2(R[2, 1], R[2, 2])

    row_data['head_pitch'] = pitch
    row_data['head_yaw'] = yaw
    row_data['head_roll'] = roll
```

I didn't know does AI work right because of my bad math, but let it go for now.

Then I paste the structure of dataset, ask AI to handle them:

Training SVM Classifier

After extracting all features from the videos, I need to train a classifier. I asked AI to write a training script for me.

I told AI that I want to use SVM with RBF kernel, and I need to:

- Filter out frames where eyes are closed
- Split data into training and testing sets
- Perform hyperparameter tuning with GridSearchCV
- Evaluate the model with 7 metrics (Confusion Matrix, Accuracy, Precision, Recall, F1 Score, ROC Curve, AUC)

AI wrote the complete `train_svm.py` script for me, which includes:

- Data loading and preprocessing
- Feature scaling with StandardScaler
- GridSearchCV for hyperparameter tuning (C and gamma)
- Model evaluation with all 7 metrics
- Visualization generation (confusion matrix and ROC curve)
- Model saving

The script worked very well and achieved 97.7% accuracy on the test set.

Real-time Testing Program

I was curious whether the model was overfitting, so I asked AI to write a real-time webcam prediction program.

I asked AI to create a program that:

- Loads the trained model and scaler
- Captures frames from webcam
- Extracts features in real-time using MediaPipe
- Predicts whether I'm watching the screen
- Displays the result with a nice UI

AI wrote the complete `realtime_prediction.py` script for me, which includes:

- Real-time face detection and feature extraction
- Prediction with smoothing (moving average)
- Visual feedback with color-coded status
- Probability display
- Keyboard controls (quit, screenshot, reset)

The real-time program works really well and confirmed that the model is not overfitting. It correctly detects when I'm watching the screen or looking away, even when my head is turned but my eyes compensate to look at the screen.